PRÁCTICA
Conociendo los datos con Pandas y NumPy

1. Presentación

Esta práctica constituye un primer contacto con un conjunto de datos correspondiente a un escenario de **e-commerce**. El propósito inicial no será limpiar los datos ni entrenar un modelo de Machine Learning, sino aprender a observar, explorar y comprender un dataset antes de tomar decisiones sobre su procesamiento.

En un proyecto real de análisis de datos, no es conveniente comenzar aplicando transformaciones de manera automática. Antes es necesario conocer qué información contiene el conjunto de datos, cómo está organizada, qué tipos de variables presenta y qué características pueden resultar relevantes para su posterior utilización. Por este motivo, la primera etapa de esta práctica estará centrada en la exploración de los datos.

Para realizar esta exploración se utilizarán dos bibliotecas fundamentales del ecosistema Python: Pandas y NumPy. La práctica no busca únicamente aprender una serie de instrucciones o comandos, sino comprender qué problema resuelve cada biblioteca y por qué ambas suelen utilizarse de manera complementaria en el análisis y preparación de datos.

El archivo de trabajo será **ecommerce_raw.csv**. A partir de este archivo, los estudiantes deberán utilizar las herramientas proporcionadas por Pandas y NumPy para responder preguntas como: ¿qué contiene el dataset?, ¿cómo está estructurado?, ¿qué tipos de datos presenta?, ¿qué variables posee?, ¿existen valores faltantes?, ¿hay registros repetidos?, ¿las variables numéricas presentan comportamientos inesperados?

El dataset pertenece al contexto de un e-commerce, pero la metodología aprendida durante esta práctica puede aplicarse a prácticamente cualquier conjunto de datos que posteriormente vaya a ser utilizado para análisis, visualización o Machine Learning.

Como alternativa, podrán proponer un dataset diferente para realizar la práctica, siempre que este cumpla con las condiciones establecidas y permita aplicar las mismas técnicas de exploración y análisis.

2. Objetivos

Al finalizar, se deberá poder cargar un CSV en Google Colab, crear un DataFrame, reconocer filas y columnas, interpretar dimensiones e instancias, inspeccionar tipos de datos, medir el tamaño del archivo y del objeto en memoria, detectar valores faltantes y duplicados, analizar variables categóricas y obtener estadísticas descriptivas.

También deberá repetir varias operaciones con NumPy y comparar ambas herramientas. El objetivo es comprender que Pandas está especialmente orientado al análisis tabular, mientras que NumPy proporciona estructuras y operaciones fundamentales para la computación numérica.

3. El dataset de trabajo

Cada fila representa una interacción de un usuario con un producto dentro del e-commerce. La tabla combina información del usuario, del producto, del contexto de interacción y del resultado.

Ejemplos de variables:

interaccion_id	identificador de interacción
usuario_id	identificador de usuario
producto_id	identificador de producto
fecha	fecha
hora	hora
edad	edad del usuario
provincia	provincia
categoria	categoría del producto
subcategoria	subcategoría
marca	marca
precio_base	precio antes del descuento
descuento	descuento aplicado
precio_pagado	precio final
rating	valoración
stock	stock disponible
visitas_previas	visitas anteriores
compras_previas	compras anteriores
agrego_carrito	indicador de carrito
compra	indicador de compra
cantidad	unidades compradas
dispositivo	dispositivo utilizado

4. Preparar Google Colab

Crear un notebook nuevo, subir ecommerce_raw.csv desde el panel de archivos e importar las bibliotecas.

```
import pandas as pd
import numpy as np
```

Pandas se utilizará principalmente para tablas. NumPy se utilizará para arrays, vectores, matrices y operaciones matemáticas.

5. Cargar el CSV y comprender el delimitador

```
df = pd.read_csv("ecommerce_raw.csv", sep=",")
```

Nuestro archivo utiliza la coma como delimitador. El delimitador separa una columna de otra dentro del archivo de texto.

```
# Si el archivo utilizara punto y coma:
df = pd.read_csv("archivo.csv", sep=";")
```

Una lectura con un separador incorrecto puede provocar que todo el contenido aparezca como una sola columna.

6. Primera inspección

```
df
df.head()
df.head(10)
df.tail()
```

head() permite observar las primeras filas y tail() las últimas. Esta inspección sirve para reconocer la estructura y detectar problemas evidentes.

7. Filas, columnas e instancias

7.1 Pandas

```
df.shape
df.shape[0]
df.shape[1]
len(df)
```

Se espera (12010, 21). La primera cifra representa las filas o instancias y la segunda las columnas o variables.

7.2 NumPy

```
array = df.to_numpy()
array.shape
array.shape[0]
array.shape[1]
```

NumPy también conoce la forma de un array. La diferencia es que un ndarray convencional no conserva la semántica de tabla del mismo modo que un DataFrame.

8. Nombres de variables

8.1 Pandas

```
df.columns
list(df.columns)
```

8.2 NumPy

```
array[:, 0]
array[:, 1]
```

Pandas permite seleccionar una variable por nombre. En un array convencional las columnas se identifican por posición.

9. Seleccionar una columna

9.1 Pandas

```
edades = df["edad"]
edades.head()
```

9.2 NumPy

```
edades_np = array[:, 5]
edades_np[:5]
```

Ambas operaciones pueden obtener una columna. Pandas resulta más expresivo porque el nombre de la variable forma parte de la operación.

10. Tipos de datos

10.1 Pandas

```
df.dtypes
df.info()
```

Pandas conserva información de tipo para cada columna. Esto es importante porque el dataset es heterogéneo.

10.2 NumPy

```
array.dtype
```

Un ndarray convencional trabaja normalmente con un dtype común para sus elementos. Al convertir una tabla heterogénea a NumPy puede ser necesario utilizar un tipo común que no sea tan útil para el análisis tabular.

Nota: El tipo técnico de una columna y su significado conceptual no son exactamente la misma cuestión.

11. Estadística básica

11.1 Pandas

```
df["edad"].mean()
df["edad"].min()
df["edad"].max()
df["edad"].std()
```

11.2 NumPy

```
np.mean(df["edad"])
np.min(df["edad"])
np.max(df["edad"])
np.std(df["edad"])
```

Las dos bibliotecas permiten estos cálculos. NumPy proporciona operaciones matemáticas fundamentales y Pandas las integra naturalmente sobre Series y DataFrames.

Nota. Algunas funciones estadísticas pueden utilizar convenciones diferentes, por ejemplo respecto de grados de libertad. Hay que conocer qué calcula exactamente cada función.

12. Estadística descriptiva

12.1 Pandas

```
df.describe()
```

describe() resume count, mean, std, min, cuartiles y max para variables numéricas.

12.2 NumPy

```
np.mean(df["edad"])
np.std(df["edad"])
np.min(df["edad"])
np.percentile(df["edad"], 25)
np.median(df["edad"])
np.percentile(df["edad"], 75)
np.max(df["edad"])
```

Con NumPy podemos construir manualmente el mismo análisis.

13. Valores faltantes

13.1 Pandas

```
df.isnull()
df.isnull().sum()
df.isnull().sum().sum()
```

El primer comando genera una máscara booleana, el segundo cuenta faltantes por columna y el tercero calcula el total.

13.2 NumPy

```
np.isnan(df["edad"])
```

NumPy permite detectar NaN de forma directa en datos numéricos. En un dataset heterogéneo, Pandas ofrece una solución más natural para el diagnóstico general.

14. Variables categóricas

14.1 Pandas

```
df["categoria"].unique()
df["categoria"].nunique()
df["categoria"].value_counts()
```

unique() devuelve valores diferentes, nunique() cuenta valores diferentes y value_counts() muestra frecuencias.

14.2 NumPy

```
valores, cantidades = np.unique(
   df["categoria"], return_counts=True
)
dict(zip(valores, cantidades))
```

NumPy también puede hacerlo, pero requiere más pasos.

15. Tamaño del archivo y memoria

```
import os

tamaño_bytes = os.path.getsize("ecommerce_raw.csv")
print("Bytes:", tamaño_bytes)
print("KB:", tamaño_bytes / 1024)
print("MB:", tamaño_bytes / 1024**2)
```

El tamaño corresponde al archivo en disco y no debe confundirse con la memoria ocupada por el DataFrame.

```
df.info(memory_usage="deep")
```

16. Duplicados

```
df.duplicated().sum()
```

Permite conocer cuántas filas completas aparecen duplicadas. En esta etapa solamente se detectan: no se eliminan.

17. Primer diagnóstico

```
print("Filas:", df.shape[0])
print("Columnas:", df.shape[1])
print("Celdas:", df.shape[0] * df.shape[1])
print("Valores faltantes:", df.isnull().sum().sum())
print("Filas duplicadas:", df.duplicated().sum())
```

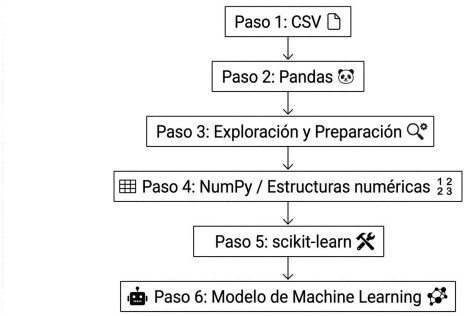
Este bloque proporciona una primera fotografía cuantitativa del dataset. Es un diagnóstico inicial, no un proceso de limpieza.

18. Pandas vs NumPy: comparación

Tarea	Pandas	NumPy
Dimensiones	df.shape	array.shape
Filas	df.shape[0]	array.shape[0]
Columnas por nombre	df["edad"]	No directamente
Primeras filas	df.head()	array[:5]
Tipos por columna	df.dtypes	array.dtype
Promedio	Series.mean()	np.mean()
Mínimo / máximo	min() / max()	np.min() / np.max()
Desviación	std()	np.std()
Resumen estadístico	describe()	Funciones individuales
Valores faltantes	isnull()	np.isnan() para numéricos
Valores únicos	unique()/nunique()	np.unique()
Frecuencias	value_counts()	np.unique(..., return_counts=True)
Datos tabulares	Muy natural	Posible, menos natural
Arrays y matrices	Muy bueno	Excelente

19. ¿Por qué usamos ambas?

No se trata de decidir que Pandas es mejor que NumPy. Pandas aporta estructuras tabulares con nombres, índices y operaciones de análisis. NumPy aporta arrays eficientes y operaciones numéricas vectorizadas.



En un proyecto real el flujo puede variar. Lo importante es comprender que las bibliotecas se integran.

20. Reseña de Pandas

Pandas es una biblioteca de análisis y manipulación de datos para Python. Su estructura central, el DataFrame, representa datos organizados en filas y columnas y permite trabajar con diferentes tipos de información dentro de una misma tabla. Es especialmente útil para limpieza, transformación, filtrado, agrupamiento, manejo de fechas y análisis exploratorio.

21. Reseña de NumPy

NumPy es una biblioteca fundamental para la computación científica en Python. Su estructura principal es el ndarray, un array multidimensional que permite realizar operaciones vectorizadas de manera eficiente. Es especialmente importante para vectores, matrices, estadística, álgebra lineal y cálculos numéricos.

22. Regla práctica

Como orientación inicial, cuando la pregunta sea '¿cómo trabajo con esta tabla, sus columnas y sus valores faltantes?', Pandas suele ser la opción natural. Cuando la pregunta sea '¿cómo realizo esta operación matemática sobre un vector o matriz?', NumPy suele ser la herramienta natural. No es una frontera rígida: ambas se complementan.

23. Actividad integradora

Resolver los siguientes desafíos en un notebook de Google Colab. Cada ejercicio debe contener el código y una breve interpretación cuando se solicite. No modificar definitivamente el dataset RAW.

1. Cargar y verificar

Suban ecommerce_raw.csv, importen Pandas y NumPy y carguen el archivo indicando explícitamente sep=",". Comprueben que el DataFrame se creó correctamente y muestren las primeras cinco filas.

Código y respuesta:

2. Dimensiones
Determinen cantidad de instancias y variables con Pandas. Luego conviertan el DataFrame a NumPy y obtengan nuevamente sus dimensiones. Expliquen por qué coinciden.

Código y respuesta:

3. Tamaño del archivo
Calculen el tamaño del CSV en bytes, KB y MB. Luego obtengan el uso de memoria mediante df.info(memory_usage="deep"). Expliquen por qué ambos tamaños no tienen por qué ser iguales.

Código y respuesta:

4. Inventario de variables
Muestren todos los nombres de columnas. Clasifiquen las variables en identificadores, numéricas, categóricas, temporales y binarias.

Código y respuesta:

5. Tipos de datos
Ejecuten df.dtypes e interpreten al menos cinco columnas. Identifiquen alguna variable cuyo tipo técnico deba investigarse.

Código y respuesta:

6. Pandas vs NumPy: selección
Seleccionen edad mediante Pandas y luego mediante NumPy. Comparen la legibilidad y expliquen qué información conserva Pandas.

Código y respuesta:

7. Estadística básica
Calculen promedio, mínimo, máximo y desviación estándar de edad con Pandas. Repitan con NumPy y comparen.

Código y respuesta:

8. Estadística completa
Ejecuten df.describe(). Reproduzcan con NumPy mínimo, Q1, mediana, Q3 y máximo de edad. Expliquen los cuartiles.

Código y respuesta:

9. Valores faltantes
Calculen nulos por columna y total. Identifiquen las tres variables con más faltantes. No los eliminen.

Código y respuesta:

10. Categorías
Obtengan valores únicos, cantidad de únicos y frecuencias de categoria. Repitan con NumPy.

Código y respuesta:

11. Dispositivos
Analicen el dispositivo. Detecten todos los valores diferentes. ¿Todos parecen válidos? No corrijan.

Código y respuesta:

12. Rangos sospechosos
Analicen edad, rating, descuento y precio_base. Obtengan mínimos y máximos. Registren resultados que deberían investigarse.

Código y respuesta:

13. Duplicados
Determinen cuántas filas completas están duplicadas. Investiguen al menos un duplicado. No lo eliminen.

Código y respuesta:

14. Variable binaria
Analicen compra y agrego_carrito. Obtengan valores únicos y frecuencias. Comprueben si aparecen valores distintos de 0 y 1.

Código y respuesta:

15. NumPy y vectorización
Construyan un array con cinco edades y sumen 10 a todas usando una sola operación de NumPy. Expliquen por qué esta forma es importante.

Código y respuesta:

16. Diagnóstico de una variable
Elijan una variable numérica e informen tipo, válidos, faltantes, media, mediana, mínimo, máximo y desviación estándar. Indiquen qué debería investigarse.

Código y respuesta:

17. Comparación argumentada
Construyan una tabla con al menos ocho operaciones e indiquen si resulta más natural Pandas, NumPy o cualquiera. Justifiquen cinco elecciones.

Código y respuesta:

18. Informe final
Redacten 10 a 15 líneas: ¿cómo es el dataset?, ¿qué problemas detectaron?, ¿qué desconocen?, ¿por qué no deben limpiar sin analizar?, ¿qué papel tendrán Pandas y NumPy?

Código y respuesta:

24. Preguntas de reflexión
¿Tener 12.010 filas significa automáticamente tener 12.010 usuarios?

¿Una columna numérica siempre debe entrar directamente a un modelo?

¿Un valor faltante es necesariamente un error?

¿Un valor extremo es siempre un outlier que debe eliminarse?

¿Por qué no conviene reemplazar todos los valores faltantes con cero?

¿Por qué una fecha almacenada como texto puede ser problemática?

¿Qué diferencia existe entre detectar un problema y solucionarlo?

¿Por qué el mismo cálculo puede ser posible tanto con Pandas como con NumPy?

25. Qué NO hacer todavía

No eliminar duplicados, outliers ni valores faltantes. No reemplazar categorías. No convertir automáticamente todas las variables a números. No normalizar ni estandarizar. No seleccionar variables para un modelo. Esta práctica termina en el diagnóstico.

26. Resultado esperado

El resultado es un notebook que documenta el estado inicial del dataset. Debe poder responder cuántas instancias y variables existen, qué tipos de datos hay, cuánto ocupa el archivo, cuántos valores faltantes y duplicados se detectan, qué categorías aparecen y qué valores merecen investigación.

La etapa siguiente será transformar este diagnóstico en un proceso de calidad de datos: tratamiento de faltantes, corrección de formatos, detección de inconsistencias, análisis de outliers y preparación de variables para Machine Learning.

27. Referencias y reseñas recomendadas

McKinney, Wes. Python for Data Analysis. O'Reilly Media. Referencia central para Pandas, NumPy, exploración, limpieza y manipulación de datos.

Harris, Charles R. et al. Array programming with NumPy. Nature, 2020. Referencia académica sobre NumPy y el paradigma de programación con arrays.

VanderPlas, Jake. Python Data Science Handbook. O'Reilly Media. Presenta de forma integrada NumPy, Pandas, visualización y herramientas de ciencia de datos.

Documentación oficial de Pandas y NumPy. Se recomienda utilizarla como referencia técnica para verificar el comportamiento exacto de las funciones utilizadas.